

Documentação do Projeto de Aprendizado de Máquina

Implantação

1. Visão Geral

A fase de implantação do projeto tem como objetivo disponibilizar o modelo de previsão de inundações de forma acessível e segura, permitindo que usuários consultem o risco de inundações mensais em diferentes províncias de Angola.

O modelo de aprendizado de máquina foi desenvolvido, treinado e avaliado localmente, e posteriormente integrado a uma aplicação web construída com **Flask**, que expõe uma **API REST** e uma **interface web interativa**.

O sistema utiliza **dados meteorológicos e hidrológicos da API da NASA** combinados com **dados manuais selecionados por usuários** através de um **mapa interativo**.

A implantação será inicialmente feita **em ambiente local**, com o objetivo de evoluir para uma hospedagem **na plataforma PythonAnywhere**, garantindo escalabilidade e disponibilidade pública.

2. Serialização de Modelos

Após o treinamento, o modelo foi **serializado utilizando o módulo pickle**, uma biblioteca padrão do Python que permite a conversão de objetos Python (como modelos treinados) em um formato binário para armazenamento e posterior carregamento.

Processo de serialização:

- O modelo treinado foi salvo em disco utilizando:

```
import pickle

with open(model_filename, 'wb') as f:
    pickle.dump(model, f)
```

- O arquivo serializado é carregado pela API no momento da inicialização do servidor Flask:

```
import pickle

with open(model_filename, 'rb') as f:
    model = pickle.load(f)
```

- O modelo é armazenado de forma **segura no servidor local** e **não é exposto publicamente via endpoints**, evitando qualquer tipo de **vazamento do modelo**.

Considerações:

- O formato **.pkl (pickle)** foi escolhido por ser simples, amplamente compatível e nativo do Python, o que facilita a integração com o ambiente Flask.
- O arquivo é armazenado em **diretórios internos inacessíveis ao público**, garantindo segurança contra acesso indevido.
- Em versões futuras, poderá ser adotado **armazenamento em nuvem criptografado** para facilitar a atualização e distribuição segura do modelo.

3. Modelo de Serviço

O modelo serializado é carregado e utilizado por um **servidor Flask** que processa solicitações de previsão.

A arquitetura de serviço foi dividida em duas camadas principais:

1. **Camada de API (Flask REST)** – responsável por receber requisições, processar os dados de entrada e retornar as previsões.
2. **Camada Web (Interface Gráfica)** – desenvolvida com HTML, CSS e JavaScript, permite interação via navegador e exibe resultados visualmente, incluindo um **mapa interativo de Angola**.

Ambiente de implantação:

- **Ambiente local:** servidor Flask executado localmente (`localhost:5000`).
- **Ambiente futuro:** hospedagem em **PythonAnywhere**, que permite execução contínua de aplicações Flask com suporte HTTPS.

4. Integração de API

O modelo foi integrado a uma **API Flask** para permitir consultas programáticas e alimentação da interface web.

Principais endpoints:

1. `POST /api/predict`
 - **Descrição:** recebe dados meteorológicos e retorna a probabilidade de inundação.

- **Entrada (JSON):**

```
{
  "precipitacao": 120.5,
  "umidade": 75.2,
  "temperatura": 28.4,
  "provincia": "Luanda"
}
```

- **Saída (JSON):**

```
{
  "provincia": "Luanda",
  "probabilidade_inundacao": 0.82,
  "risco": "Alto"
}
```

2. GET /api/provincias

- **Descrição:** retorna uma lista de províncias e seus riscos de inundação calculados automaticamente.
- **Saída (JSON):**

```
[
  { "provincia": "Luanda", "risco": "Alto" },
  { "provincia": "Benguela", "risco": "Médio" },
  { "provincia": "Huambo", "risco": "Baixo" }
]
```

3. GET /

- **Descrição:** página principal da aplicação

Integrações externas:

- API da NASA: fornece dados de precipitação e outros indicadores ambientais atualizados automaticamente.

5. Considerações de Segurança

Diversas medidas foram adotadas para garantir a segurança da aplicação e do modelo:

- **Criptografia HTTPS:** todas as comunicações entre o cliente e o servidor ocorrem via HTTPS, evitando interceptação de dados sensíveis.
- **Isolamento do modelo:** o arquivo do modelo não é exposto nem transferido através da API; apenas as previsões são retornadas.
- **Sanitização de entradas:** todas as requisições são validadas antes do processamento para prevenir ataques de injeção.
- **Autenticação básica (futura):** será implementada autenticação via tokens JWT para controlar o acesso à API em ambiente de produção.
- **Controle de logs:** os logs do sistema não armazenam dados sensíveis enviados pelos usuários.

6. Monitoramento e Registro

O desempenho e o uso da aplicação são monitorados continuamente por meio de logs e métricas locais.

Monitoramento:

- Uso de **logs Flask** para registrar:
 - Requisições de API.
 - Erros de execução.
 - Tempo de resposta do modelo.
- As métricas de desempenho do modelo (como **acurácia e precisão**) são recalculadas periodicamente usando novos dados coletados.

Registro e alertas:

- Logs armazenados localmente em arquivos `.log`.
- Em versões futuras (na hospedagem PythonAnywhere), pretende-se:
 - Integrar com ferramentas de monitoramento (como Grafana ou Prometheus).
 - Configurar **alertas por e-mail** para falhas ou tempos de resposta anormais.